CINEMATCH

Hybrid Movie Recommendation System

PROJECT REPORT

A movie discovery application that combines movie similarity, viewer rating patterns, and popularity to rank recommendations.

Project Overview

CineMatch is a movie discovery web application designed to help users search a movie catalogue, find similar titles, receive personalized suggestions, adjust recommendation preferences, and maintain a watchlist. The application combines movie attributes and rating patterns to produce ranked results.

Project name	CineMatch — Hybrid Movie Recommendation System
Web interface	Next.js and React
Application API	FastAPI
Data processing	Python, Pandas, NumPy and SciPy
Movie data	MovieLens ml-latest-small

1. Motivation

A large movie catalogue can make it difficult for viewers to identify films that match their interests. Searching by title alone does not help a viewer discover related films, while a list ordered only by average rating may repeatedly show the same popular titles. CineMatch addresses this problem by combining information about movie genres and tags with patterns in user ratings. The application presents the resulting suggestions through a guided interface so that users can explore, refine, and save titles in one place.

2. Objective

- Build a web-based application for searching and exploring a movie catalogue.
- Recommend movies similar to a selected title using genres and community tags.
- Generate personalized recommendations from user ratings and viewing preference patterns.
- Combine content similarity, collaborative preference scores, and popularity into a common ranking.
- Provide filters and clear explanations that help users understand why a movie appears in the results.
- Allow users to maintain a session-based watchlist and adjust recommendation preferences.
- Evaluate recommendation quality on held-out ratings and compare the hybrid approach with baseline methods.

3. Workflow

The application follows a sequence from data preparation to recommendation display. MovieLens CSV files are validated, cleaned, and transformed into a movie catalogue and rating data structures. The recommendation components generate candidate titles and scores. The application API exposes search and recommendation operations to the web interface.

Stage	Description
1. Data input	Load movies.csv, ratings.csv, tags.csv and links.csv from the project data directory.
2. Validation	Check required columns, missing values, duplicate rows, rating ranges, and movie ID consistency.
3. Feature preparation	Combine genres and tags for content matching; calculate rating statistics and prepare rating interactions.
4. Content recommendations	Represent movie text using TF-IDF and compare movies with cosine similarity.
5. Collaborative recommendations	Use patterns in observed user ratings with item-neighbour and matrix-factorization methods.
6. Hybrid ranking	Normalize available signals, combine them with popularity, apply filters, and exclude ineligible titles.
7. Presentation	Show ranked movie cards with genres, ratings, explanations, and watchlist actions.
8. Evaluation	Use a temporal train/validation/test split to assess rating prediction and ranking quality.

3.1 Application workflow

- Discover: search the catalogue and explore popular titles.
- Similar Movies: select a movie, adjust filters, and view related titles.
- For You: rate selected movies or choose a labelled sample profile to generate personalized suggestions.
- Tune: adjust novelty, diversity, and era preferences to refine results.
- Library: save or remove movies from the current watchlist.
- Model information: review dataset details and evaluation summaries.

3.2 System architecture

The current project archive contains a Next.js/React web client and a FastAPI service. The web client manages the movie discovery scenes, preference selection, recommendation controls, movie details, and saved library. The API loads the catalogue and recommendation engine, handles requests, and returns structured movie cards. Python modules handle data validation, feature preparation, content matching, collaborative scoring, hybrid ranking, and evaluation.

Layer	Main components	Responsibility
Interface	Next.js, React, Tailwind CSS, GSAP, Motion	Search, onboarding, controls, movie cards and transitions
API	FastAPI	Search, similar titles, personalized recommendations and model metadata
Recommendation core	Python, NumPy, SciPy, scikit-learn	Feature processing, content similarity, collaborative scoring and ranking
Data	MovieLens CSV files and local database	Movie metadata, user ratings, tags and ID mappings

3.3 Dataset and features

The supplied project archive contains MovieLens CSV files with 9,742 movie records, 100,836 rating records, 3,683 tag records, and 9,742 link records. The ratings represent 610 distinct users and range from 0.5 to 5 stars.

File	Fields used	Purpose
movies.csv	movieId, title, genres	Movie identity and descriptive categories
ratings.csv	userId, movieId, rating, timestamp	Preference history and evaluation
tags.csv	userId, movieId, tag, timestamp	Additional text signals for content matching
links.csv	movieId, imdbId, tmdbId	External identifier mapping

Archive inspection found 0 rating rows and 0 tag rows referencing unknown movie IDs. There are 18 catalogue entries without a rating in ratings.csv. These checks are based on the supplied CSV files; they do not imply that every movie has complete descriptive metadata.

3.4 Feature engineering and ranking

Feature	Transformation / role
Genres and tags	Clean and combine genre labels and aggregated tags into text features.
TF-IDF representation	Convert combined text into sparse numerical vectors.
Content similarity	Use cosine similarity to find movies with related genre/tag descriptions.
Rating interactions	Use observed user–movie ratings to estimate preference patterns.
Popularity statistics	Calculate rating averages and support counts, with smoothing to reduce the influence of low-count titles.
Hybrid score	Combine normalized content, collaborative, and popularity signals, with fallback behavior when signals are unavailable.

The implementation includes item-neighbour collaborative filtering and a biased matrix-factorization approach. For new or lightly rated profiles, the system can use content and popularity signals while preference evidence is limited. The final score is normalized across available signals; weights and serving strategy are selected through validation rather than assumed to be universally optimal.

4. Code

The codebase separates the user interface, API, data preparation, recommendation logic, and evaluation. The following examples summarize the principal operations represented in the project; they are explanatory excerpts, not a replacement for the complete source files.

4.1 Data loading and validation

```
movies = pd.read_csv(data_dir / "movies.csv")
ratings = pd.read_csv(data_dir / "ratings.csv")
tags = pd.read_csv(data_dir / "tags.csv")
links = pd.read_csv(data_dir / "links.csv")

# Validate required columns and match movie IDs
assert {"movieId", "title", "genres"} <= set(movies.columns)
assert {"userId", "movieId", "rating", "timestamp"} <= set(ratings.columns)
unknown_rating_ids = ~ratings["movieId"].isin(movies["movieId"])
```

4.2 Content feature preparation

```
# Aggregate tags and combine with genres
tag_text = tags.groupby("movieId")["tag"].apply(
    lambda values: " ".join(values.astype(str))
)
catalog = movies.merge(tag_text.rename("tag_text"),
                       on="movieId", how="left")
catalog["tag_text"] = catalog["tag_text"].fillna("")
catalog["content_text"] = (
    catalog["genres"].fillna("").str.replace("|", " ", regex=False)
    + " " + catalog["tag_text"]
)

# TF-IDF features are compared using cosine similarity
X = vectorizer.fit_transform(catalog["content_text"])
```

4.3 Hybrid ranking

```
# Conceptual form of the ranker
score = (
    content_weight * normalized_content_score
    + collaborative_weight * normalized_preference_score
    + popularity_weight * normalized_popularity_score
)
# Re-normalize available weights for cold-start cases,
# apply user filters, and remove already-rated movies.
```

The full implementation is available in the supplied project source. The backend exposes endpoints for movie search, similar-movie retrieval, personalized recommendations, demo profiles, and model metadata. The web client consumes these endpoints to render the discovery workflows.

5. Demonstration/Result

5.1 Demonstration workflow

1. Launch the API service and the web application.
2. Open the Discover screen and search for a movie title.
3. Select a title and inspect the Similar Movies results.

4. Open For You, rate several movies or select a sample profile, and generate recommendations.
5. Adjust available filters and ranking controls to refine the list.
6. Save a movie to the Library and verify that it appears in the saved list.
7. Open the model information panel to review dataset and evaluation details.

The project archive contains implementation code and evaluation artifacts. The following results are transcribed from the stored evaluation file `eval_results.json`; they are offline evaluation results and should not be interpreted as a live demonstration run performed while preparing this report.

5.2 Dataset and evaluation setup

- Training ratings: 70,312
- Validation ratings: 9,818
- Held-out test ratings: 20,706
- Test users included in ranking evaluation: 591
- Rating prediction RMSE: 0.9389
- Rating prediction MAE: 0.7225

5.3 Recorded ranking results

Method	NDCG@10	Recall@10	Hit Rate@10	Catalog coverage
Popularity Baseline	0.0596	0.0347	28.8%	0.5%
Content-Only	0.0291	0.0208	20.1%	3.8%
Item-kNN CF	0.0420	0.0231	20.1%	2.8%
Explicit ALS	0.0456	0.0272	24.2%	2.6%
CineMatch Hybrid	0.0982	0.0703	43.5%	2.0%

On the held-out test set, the recorded CineMatch Hybrid NDCG@10 was 0.0982, with Recall@10 0.0703 and Hit Rate@10 43.5%. The popularity baseline achieved NDCG@10 0.0596. These results show that the hybrid approach can improve ranking over simple popularity in the recorded evaluation, while its catalogue coverage is lower than a random baseline and remains an area for improvement.

The stored validation selection chose CineMatch Hybrid over the best single model, Implicit Co-occurrence, with validation NDCG@10 0.0947 versus 0.0891. Evaluation values can vary with split and configuration. The archived documentation contains earlier benchmark summaries; this report uses the metrics in the root eval_results.json file.

5.4 Limitations

- The MovieLens metadata primarily contains titles, genres, tags, ratings, and external identifiers; it does not provide full plot summaries or complete cast information.
- The dataset reflects historical rating activity and may not represent the preferences of every new visitor.
- New users have limited rating history, so their recommendations rely more heavily on content and popularity signals.
- The watchlist and visitor preferences are session-oriented unless persistent storage is configured.
- The offline evaluation reflects a specific temporal split and does not guarantee identical performance for every user or future dataset.

6. Conclusion

CineMatch brings movie search, similarity-based discovery, personalized recommendations, preference tuning, and a saved library into one web application. It uses MovieLens movie metadata and ratings to construct content and collaborative signals, then combines available signals into a ranked list. The supplied evaluation artifacts provide measurable results for comparison with baseline methods. Further improvements can focus on broader catalogue coverage, richer movie metadata, persistent user preferences, and additional validation across different data splits.

How to Run the Project

From the project root, start the API service:

```
python -m uvicorn apps.api.main:app --host 127.0.0.1 --port 8000
```

In a second terminal, start the web application:

```
cd apps/web
npm install
npm run dev
```

Open http://localhost:3000 in a browser. These commands follow the project README; execution depends on the local Python and Node.js environments being installed.

Project Files

- `apps/web/` — web interface and interactive components.
- `apps/api/` — API routes, request schemas, sessions, and recommendation service.
- `src/` — data loading, preprocessing, recommendation models, and evaluation code.
- `data/` — MovieLens CSV files and local project data.
- `artifacts/eval_results.json` and `eval_results.json` — stored evaluation output.
- `tests/` — automated tests for project behavior.